

QuickDelivery - Arquitetura

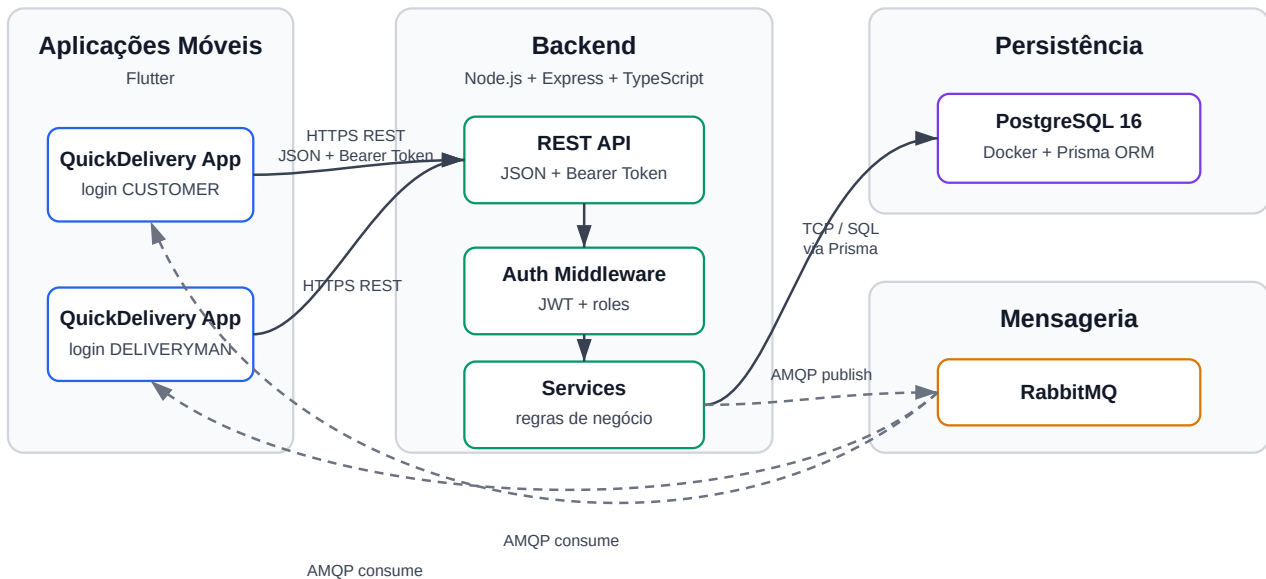


Diagrama de arquitetura do QuickDelivery

Os clientes da API consomem endpoints REST em Node.js, Express e TypeScript com payloads JSON e autenticação por Bearer Token. O backend concentra as regras de negócio e funciona como ponto único de acesso aos dados: os aplicativos não acessam diretamente o banco nem o RabbitMQ.

O middleware de autenticação valida o token JWT e disponibiliza `userId` e `role` para as regras de negócio, permitindo separar permissões de `CUSTOMER` e `DELIVERYMAN`. As validações de autorização e de transição de status ficam na camada de services.

A API persiste dados no PostgreSQL por meio do Prisma ORM. O modelo atual usa `users` e `deliveries`: clientes criam entregas próprias, entregadores visualizam entregas pendentes ou atribuídas a eles, e as transições de status são validadas antes da persistência.

O RabbitMQ é usado para a comunicação assíncrona. Eventos como `delivery.created`, `delivery.accepted` e `delivery.status_changed` são publicados via AMQP e consumidos pela fila `quickdelivery.delivery-events`, demonstrando o desacoplamento entre produtor e consumidor. No app Flutter, clientes e entregadores refletem mudanças de estado por polling periódico na API REST, enquanto o RabbitMQ permanece como MOM do backend.